

1. Equipment

- MSP-EXP430F5529LP Launchpad Board
- USB Cable (A-Male plug to Micro-B-Male plug)
- Computer/Notebook with Windows 11 or better
- Code Composer Studio (CCSTUDIO or in short CCS) Version 12.8.1

2. Learning Objective

This lab provides exercises that would allow the student to have broad overview of the basic development environment for the MSP430 microcontroller, including the use of the affordable MSP-EXP430F5529LP Launchpad board and the CCS IDE.

After this lab session, the student should be able to:

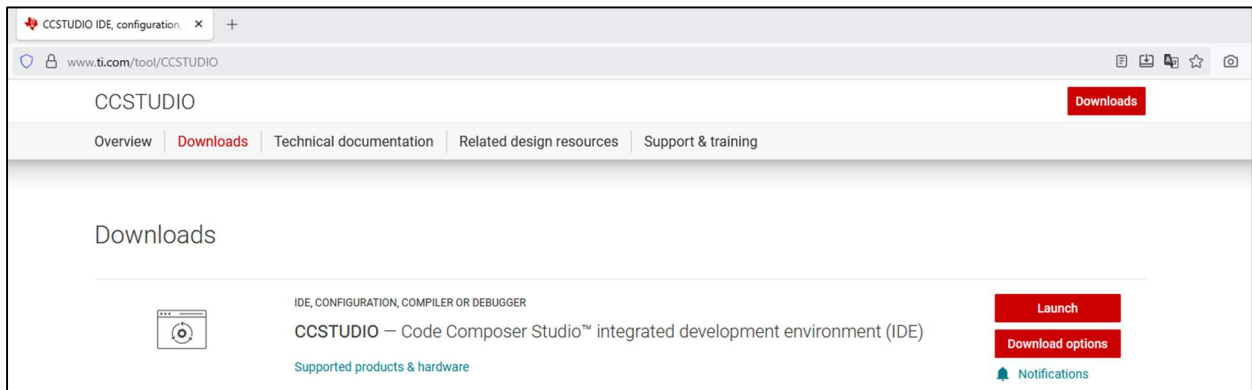
- create simple Assembly programs in the CCS IDE
- be familiar with the basic operations of the CCS IDE including editing and compiling of the source files
- use the CCS IDE debugger to emulate program execution on the MSP-EXP430F5529LP board
- utilise the debugger interface to step/execute the MSP430 program as well as view the contents of the MSP430 registers
- Understand the FETCH, DECODE and EXECUTE cycle of the MSP430 CPU

3. Overview of lab session

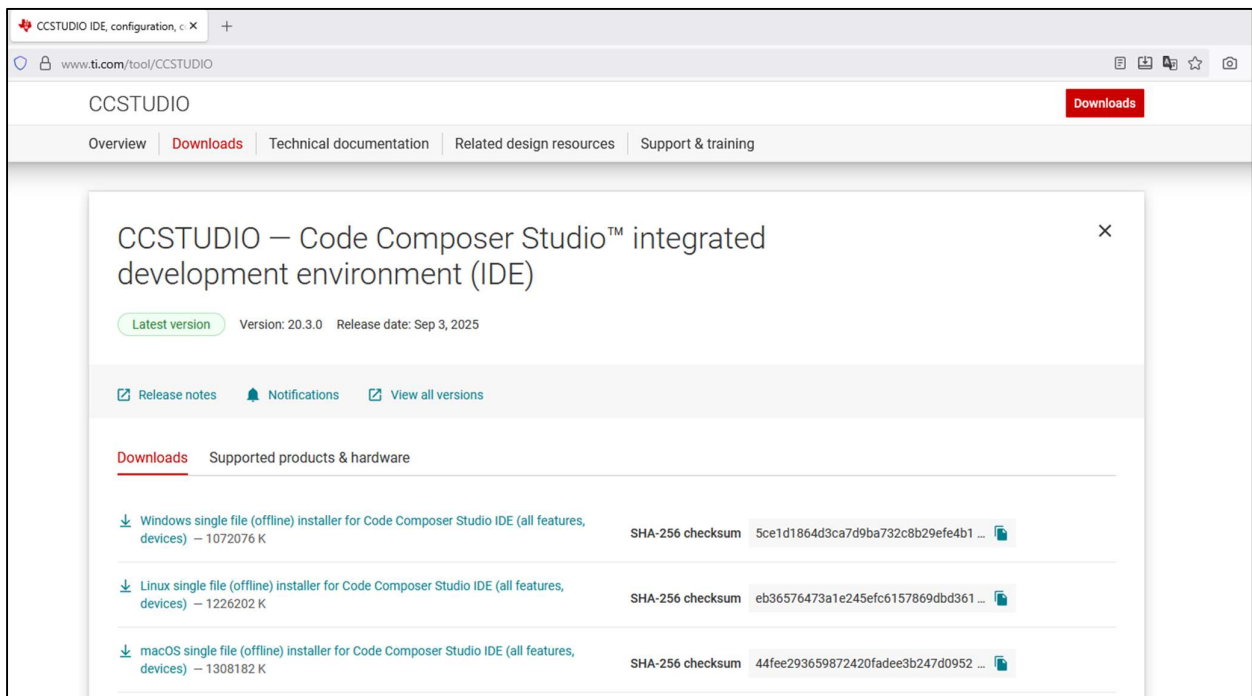
This lab session will be conducted over 2 hours and aims to familiarise students with the basic tools for creating and emulating MSP430 programs. Students are also encouraged to install the Code Composer Studio (CCSTUDIO or in short CCS) on their own machines to explore the MSP430 in their own time as there are many other exciting stuff that can be done. You can download the CCS from the following link:

<https://www.ti.com/tool/CCSTUDIO>

Please download the offline installer for CCSTUDIO (**Version 12.8.1, not version 20.x.x or 21.x.x**) based on the following steps:



Select “Download Options”



Select “View all versions”


TEXAS INSTRUMENTS

[Login / Register](#)
[English](#)
[Ship to](#)
[USD](#)

[Cross-reference search](#)

[Products](#)
[Applications](#)
[Design & development](#)
[Quality & reliability](#)
[Ordering resources](#)
[About TI](#)



[Home](#) / [Design resources](#) / [CCSTUDIO](#)

CCSTUDIO

Code Composer Studio™ integrated development environment (IDE)

Select a version

Version: 12.8.1 Release date: Sep 30, 2024

v20x

20.3.0 (Sep 3, 2025)
20.2.0 (Jun 2, 2025)
20.1.1 (Apr 1, 2025)
20.1.0 (Mar 3, 2025)
20.0.2 (Jan 23, 2025)
20.0.1 (Dec 20, 2024)
20.0.0 (Dec 2, 2024)

v12x

12.8.1 (Sep 30, 2024)
12.8.0 (Aug 9, 2024)
12.7.1 (May 10, 2024)
12.7.0 (Apr 5, 2024)
12.6.0 (Jan 12, 2024)
12.5.0 (Oct 4, 2023)
12.4.0 (Jul 10, 2023)

[Release notes](#)
[View software details](#)

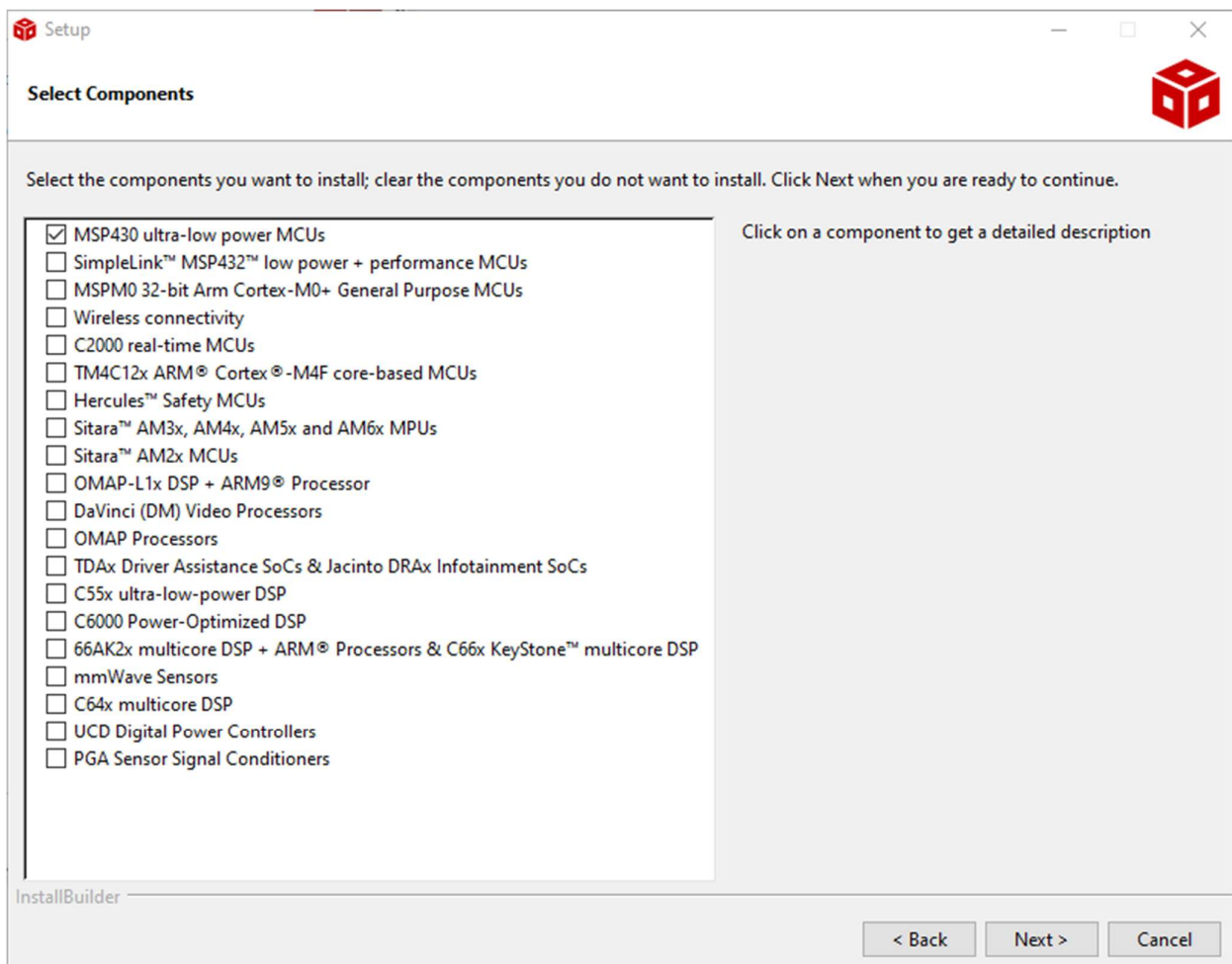
Downloads
Supported products & hardware

Windows single file (offline) installer for Code Composer Studio IDE (all features, devices) — 1350457 K	SHA-256 checksum	db6318483fb6f0358c4d65a426105818 ...
Windows on-demand (web) installer for Code Composer Studio IDE (all features, devices) — 38993 K	SHA-256 checksum	faa016454cdb641585fbcaf558f1177c4 ...
macOS single file (offline) installer for Code Composer Studio IDE (all features, devices) — 1233676 K	SHA-256 checksum	b439b6f1a6601bf379a5e28306526a48 ...
macOS on-demand (web) installer for Code Composer Studio IDE (all features, devices) — 33252 K	SHA-256 checksum	6ce12c1149a32c6a0d4148b6dd0971fa ...
Linux single file (offline) installer for Code Composer Studio IDE (all features, devices) — 1319945 K	SHA-256 checksum	7994924d77d33278d2f6f2bcef374b61 ...

Select version “12.8.1 (Sep 30, 2024)” and select the appropriate single file (offline) installer on the right based on the operating system of your computer. The installation file is over 1GB so it may take some time (up to an hour or more) to download and install so it is advised that you do this before coming to your first lab session.

Execute the file “ccs_setup_12.8.1.00005.exe” downloaded (for windows machines). During installation, please ensure that there are no spaces in the folder name where you install CCSTUDIO as that may cause issues when compiling your programs in the future.

Please select “Custom Installation” when prompted and also select “MSP430 ultra-low power MCUs” in the “Select Components” page as follows:



For this lab, students should ideally explore independently but may also interact with their fellow classmates to explore and have fun together.

The main objective of this lab session is to ensure that your MSP430 launchpad is working with your laptop so that you can try out many of the things taught in part I of the ICT1011 module.

There is also an assignment for this lab that has to be submitted via the ICT1011 module site in xSITe. The submission link can be found under the [Assessments][Quizzes] tab. Do that that this assignment is to enable you to experiment with using genAI to edit your codes through SIT's ClassAID platform. This assignment carries a weightage of 5% of your module marks so do try to do this on your own.

The Exercises

5.1. The TI MSP430 Design Work

5.1.1. Resources available

The MSP430 is a very popular modern ultra-low power systems-on-chip microcontroller that is used by a large number of industrial sectors for many different applicants. As such, it has a very large developer community with many resources and information available online. Examples of online resources includes (but not limited to):

<http://e2e.ti.com/support/microcontrollers/msp430/default.aspx>

For this lab, please download the TI MSP430 Design Workshop worksheet (Extracts for Lab 1) from xSITE. These worksheets are extracted from a larger MSP430 workshop conducted by TI.

5.1.2. Task

The relevant worksheets from the TI MSP430 Design Workshop will be used for this lab. You can also download the extracted worksheets and the sample source files **MSP-EXP430F5529LP Software Examples.zip** from xSITE if you like. It may take up to 10 minutes to build each exercise if you are doing this for the first time.

Exercise 1: Lab 2a - Creating a New CCS Project

Exercise 2: Lab 2b - My First Blinky

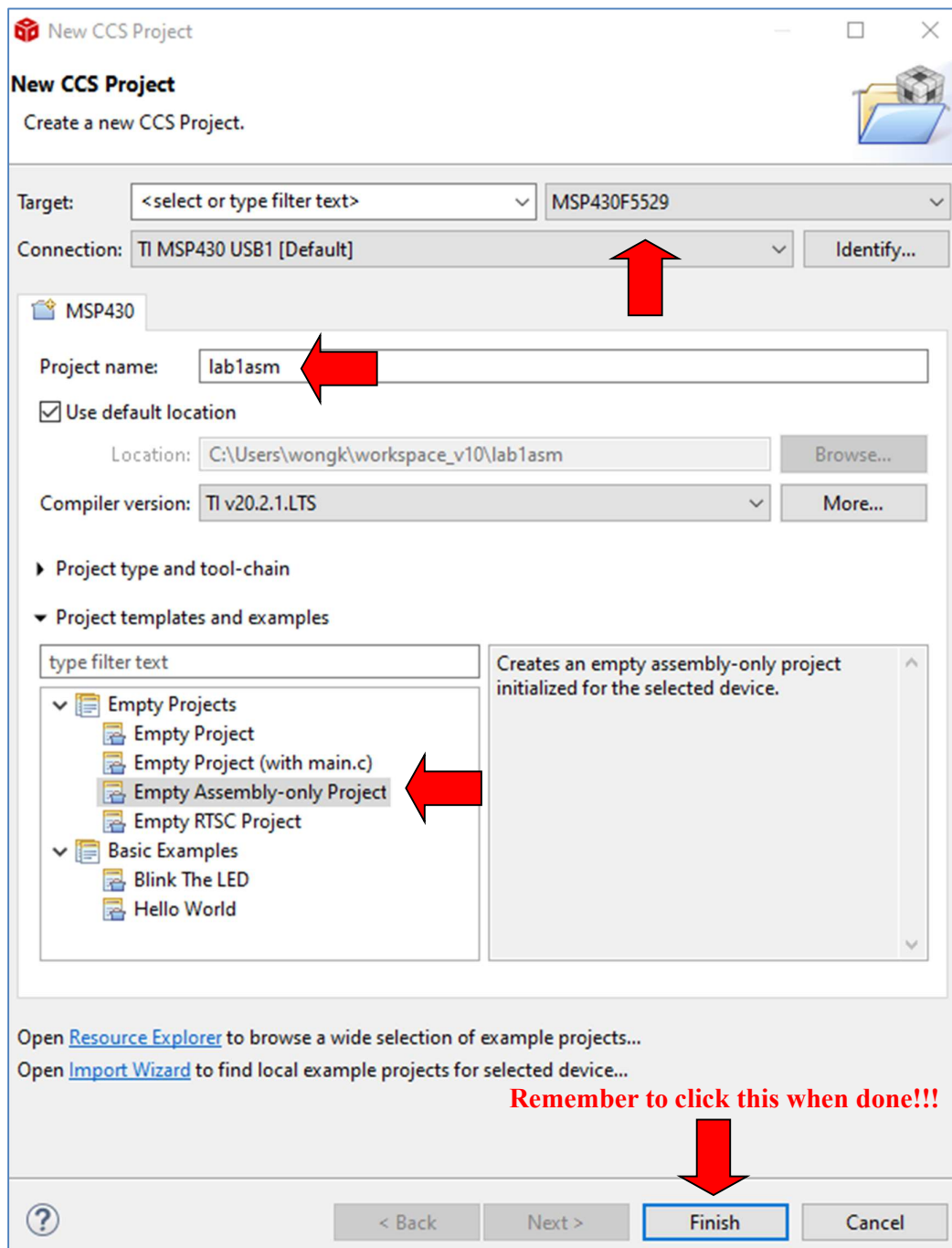
Unlike what is mentioned in the worksheet, please use the default directory as the TI workshop folder is not installed. Ignore Page 2-29 (item 1), page 2-30 (item 3), Page 2-32 (items 8d and 8e) and Page 2-34 (item 10) in the TI workshop document.

If you are having problems completing the exercises in this section, you can still proceed to do the exercises in the next section (Section 5.2) and revisit the exercises in Section 5.1 later.

5.2. Creating your 1st MSP430 Assembly Project from Scratch

In this exercise, you will be writing an assembly language version of the “My First Blinky” that you have experimented with in the previous tasks.

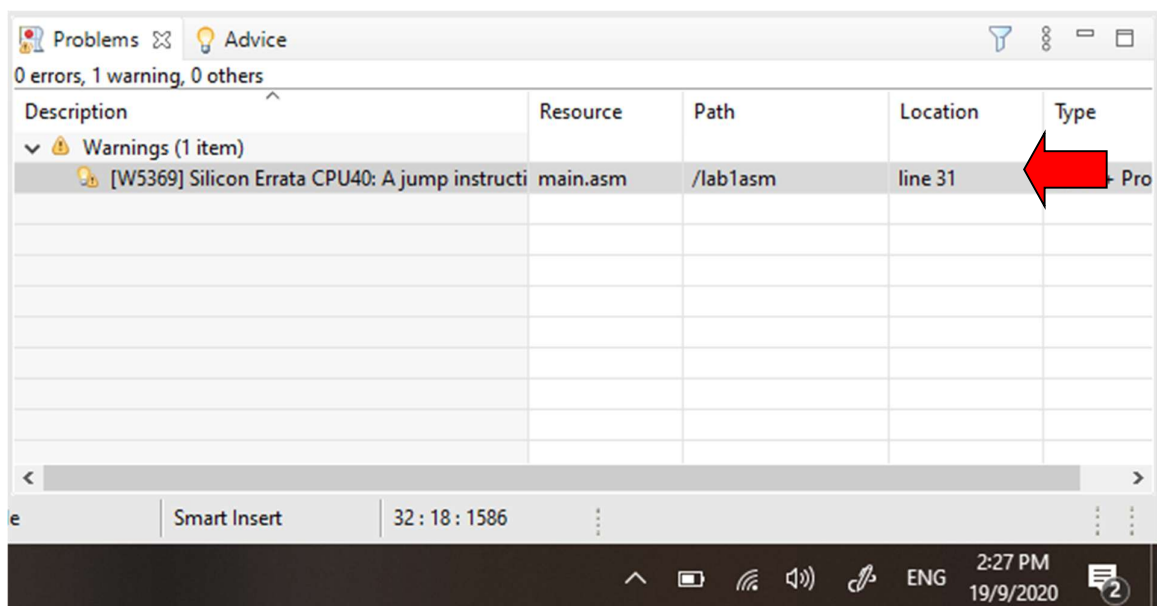
- i. Ensure that the MSP-EXP430F5529LP is connected to the computer and start the Code Composer Studio (CCS). For the latest versions of CCS, **use CCS in the “simple” mode if prompted.**
- ii. Create a new CCS Assembly-only project (*File → New → CCS Project*) for the MSP430F5529 microcontroller. Name the project “**lab1asm**”. You can ignore the compiler version as the version of code composer used might vary.



- iii. You are now “ready” to build/compile the project. In the Project Explorer window, double click on the **main.asm** file. You should see template for the source file main.asm. This is where you will be inserting the source codes for your project later.
- iv. Open the Console window (View→Console) and the Problems window (View→Problems) if they are not already opened. These windows shows the compilation process and give details of any advice, warnings or errors.

Build (compile) the program (*Project → Build all*). A warning might appear in the “Problems” window which you can ignore. Do note it will take as long as 10 minutes for the library to be built the first time.

v.



- vi. Make sure that your project can be built without any other warnings before proceeding to the next step. The “Optimization Advice” in the Problems window can be ignored.
- vii. We are now going to edit the source codes in the main.asm file to blink the LED1. Insert the following codes into the main.asm file:

```

Mainloop    bis.b #001h, &P1DIR
            xor.b #001h, &P1OUT
            mov.w #060000, R15
Delay       dec.w R15
            jnz Delay
            jmp Mainloop

```

A sample of the modified main file (with added comments) is as follows:

```
1;-----
2; MSP430 Assembler Code Template for use with TI Code Composer Studio
3;
4;
5;-----
6      .cdecls C,LIST,"msp430.h"      ; Include device header file
7;-----
8;
9      .def      RESET                ; Export program entry-point to
10     ; make it known to linker.
11;-----
12     .text                          ; Assemble into program memory.
13     .retain                          ; Override ELF conditional linking
14     ; and retain current section.
15     .retainrefs                     ; And retain any sections that have
16     ; references to current section.
17;-----
18;-----
19 RESET    mov.w    #_STACK_END,SP    ; Initialize stackpointer
20 StopWDT   mov.w    #WDTPH|WDTHOLD,&WDTCNTL ; Stop watchdog timer
21;-----
22;-----
23;-----
24; Main Loop here
25;-----
26     bis.b    #001h,&P1DIR
27 MainLoop xor.b    #001h,&P1OUT
28     mov.w    #06000h,R15
29 Delay     dec.w    R15
30     jnz      Delay
31     jmp      MainLoop
32;-----
33;-----
34;-----
35; Stack Pointer definition
36;-----
37     .global  __STACK_END
38     .sect    .stack
39;-----
40;-----
```

- viii. **Exercise 3:** Build the project. You should see a warning. Resolve this warning (Hint: You can expand the “Warnings” in the “Problems” window to see more details of the warnings)
- ix. Debug the project but do not run it yet. You should know how to do this by now.
- x. To examine the contents of the registers and the disassembled code in memory, open the register and disassembly browser. (*View→Register, View→Disassembly*)
- xi. **Exercise 4:** The Program Counter (PC) contains the address where the next instruction will be executed. Find the value of the PC using the Register Browser. This is also the address of the entry point for this program.

Value of the PC: _____

- xii. **Exercise 5:** Click on the  button once (or use Run→Step Over). This is known as single-stepping. What is the PC value now?

Value of the PC: _____

This is an example of the CPU

- (i) Fetching the instruction from the memory address pointed by PC and increment the PC value
- (ii) Decode the instruction
- (iii) Execute the instruction

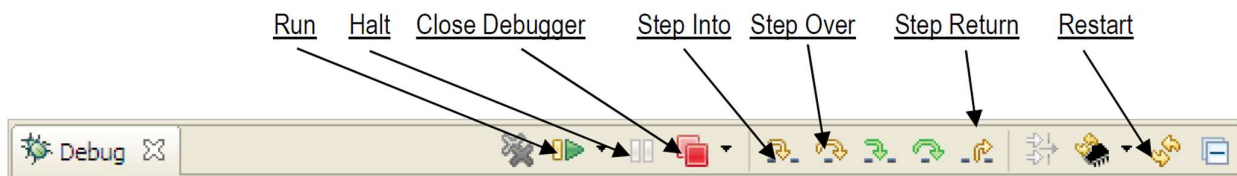
- xiii. Click on the  button (or use Run→Resume). The LED1 should now be blinking.

If you have successfully done this;

CONGRATULATIONS!!!

**You have just created your first MSP430 assembly program
(assuming that you have not done this before)!!!!**

- i. Some of the other buttons that you might use are shown below. Do try them out and see what they do.



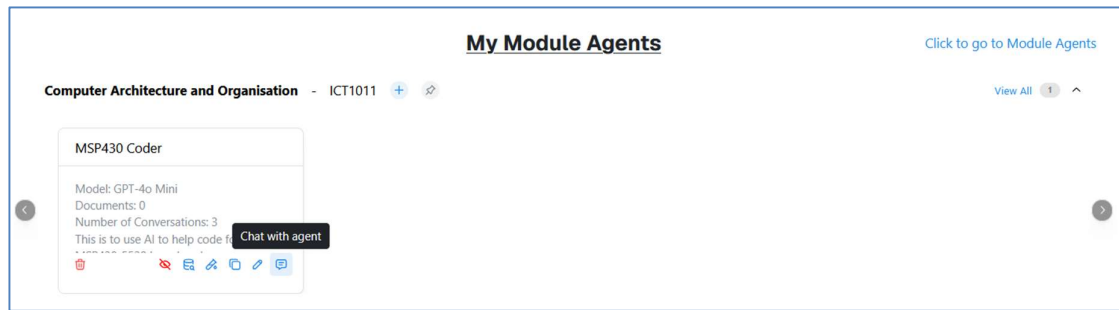
ADDITIONAL CHALLENGE 1

Modify the program

Can you make your LED blink 2 times faster?

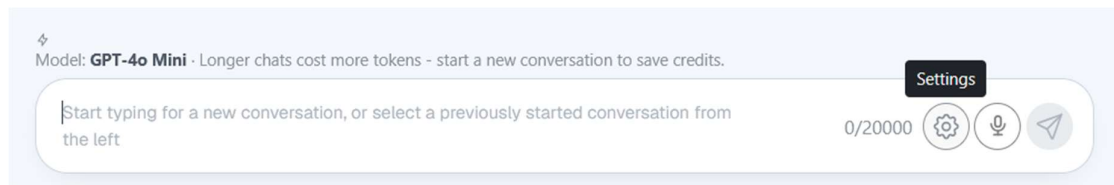
5.3. Assignment (5% of ICT1011 Marks): Exploring using genAI

- i. For this assignment, you will be using SIT's ClassAid platform which is very similar to genAI tools such as those provided by OpenAI (GPT), Google (Gemini), Anthropic(Claude), etc. ClassAid can be access through <https://classaid.singaporetech.edu.sg/home>. ClassAid allows you access to many LLMs including GPT, Gemini, Claude, etc.
- ii. You can view or recap (if you have already done it) about ClassAid as well as fundamentals of prompt engineering and vibe coding in Unit 5.2 of your **UDC1001 Digital Competency Essentials (DiCE)** module.
- iii. For this assignment, you will be using a simple MSP430 coder agent that is create in ClassAid for you to use in this exercise:



- iv. Do note that you will need to complete the quiz on the ICT1011 xSITe page. You can access the quiz using the [Assessment] tab on the xSITe page.
- v. Using the MSP430 Coder, you are to create **an MSP430 assembly language programme to blink the LED on Port 1 Pin 0** using a single prompt carried out on two different LLM models in the following categories: (a) the model with the cheapest total cost per prompt and per reply, and (b) another model of your choice with a reply cost of between 100 and 200 tokens. For the quiz, enter the following:
 - a. The LLM models you selected
 - b. The total conversation credits consumed by each model for your prompt to generate the codes
 - c. Whether the code generated worked at the first attempt

The models can be selected using the [Settings] button in the prompt textbox as follows:



- vi. This is an introductory exercise for you to explore the use of different LLMs through the SIT ClassAid platform to generate codes for your MSP430 launchpad. You will see that different models generate different results that may or may not work. Some of you may also notice that if you do not know what you are doing, AI may not always help.
- vii. You will also notice that the different models consume different cost. Do continue exploring with different models and you will realise that the most expensive models might not be the most cost effective and also may not give you the best solution.
- viii. Beyond ICT1011, you can also use ClassAid for your other modules or use it for generic chats or model comparisons through the [Others] tab. However, do note that there is a limit that is allocated for your daily use so do try to use the most credit efficient models instead of going for the newest frontier models for all your prompts.

-----End of Lab 1-----